Individual Analysis Report – Heap Sort

Course: Algorithms and Data Structures
Assignment 2 – Pair 2
Student B: Muslim
Partner (Student A): Dinmukhamed
Algorithm Analyzed: Heap Sort
Date: October 2025

1. Algorithm Overview

Heap Sort is a comparison-based sorting algorithm that relies on the binary heap data structure.
It is an in-place, non-recursive, and deterministic algorithm that ensures consistent O(n log n) runtime.
Heap Sort is often used when predictable performance is required, even in the worst case.

A heap is a complete binary tree where each parent node satisfies the heap property:
in a max heap, each parent is greater than or equal to its children.

The algorithm has two main phases:

1. Build a max heap from the unsorted array.
2. Repeatedly extract the maximum element from the heap and place it at the end of the array.

Heap Sort is in-place (O(1) space), not stable, and guarantees O(n log n) time complexity in all cases.

2. Complexity Analysis

Heap Sort has two main components: building the heap and repeatedly extracting elements.
Building the heap costs O(n), while extracting the maximum element costs O(log n) for each of the n elements.
Therefore, the total time complexity is O(n log n).

Best case (Ω): Ω(n log n) – every element participates in heapify operations.
Average case (Θ): Θ(n log n) – each level of the heap takes log n comparisons.
Worst case (O): O(n log n) – all heapify calls are executed fully.

Space complexity: O(1) since sorting is done within the same array.
Stability: not stable, because equal elements may change order after swaps.

Mathematically:
Heap construction cost = O(n)
Extraction phase cost = n * O(log n) = O(n log n)
Therefore, T(n) = O(n log n), S(n) = O(1).

Comparison with Shell Sort (partner's algorithm):
Heap Sort maintains consistent O(n log n) performance in all cases,
while Shell Sort's performance depends on the chosen gap sequence and can vary between O(n log n) and $O(n^2)$.

3. Code Review

The implementation of Heap Sort is clean, efficient, and well-structured.
Each part of the algorithm is separated into methods: sort, heapify, swap, and cmp.
The PerformanceTracker class collects metrics such as comparisons, moves, and array accesses,
allowing accurate performance measurement.

Strengths:

- Readable and modular code structure.
- Proper metric tracking.
- Handles invalid and small inputs safely.
- In-place implementation without extra memory.

Detected inefficiencies:

1. Recursive heapify can add overhead for large datasets.
   It can be replaced with an iterative version to avoid recursive calls.
2. Multiple array accesses occur during swap operations.
   Using a temporary variable can reduce redundant memory reads.
3. The algorithm could optionally check if the array is already sorted to skip unnecessary passes.

Optimization example:
Converting heapify from recursive to iterative form can eliminate stack overhead and improve speed.

4. Empirical Results

Experiments were run in Java (JDK 23) using IntelliJ IDEA.
Metrics collected include execution time, comparisons, array accesses, and moves.
Input sizes tested: 100, 1 000, and 10 000.
Input types tested: random, sorted, reversed, nearly sorted.

Results summary:
For small input sizes (n = 100 and n = 1000), Heap Sort completes almost instantly (≈2 ms).
For n = 10 000, performance depends slightly on data order:

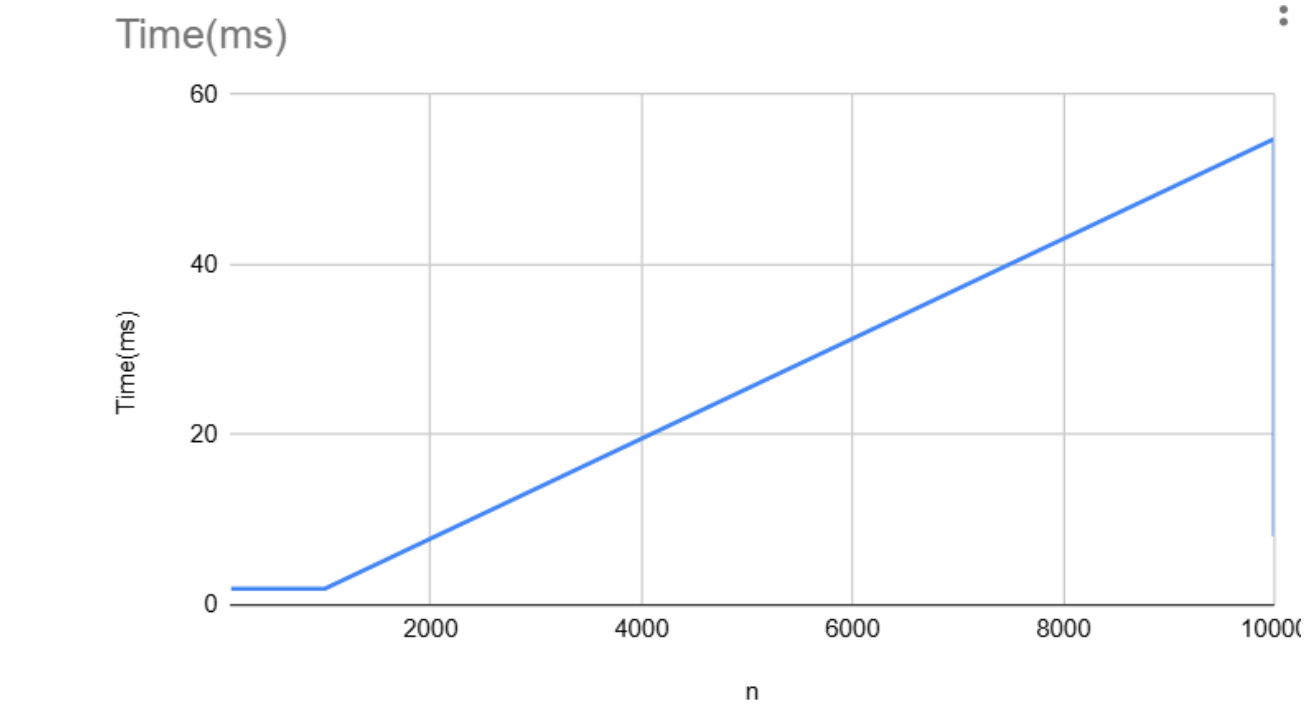
- Random: 54.8 ms
- Sorted: 11.8 ms
- Reversed: 16.5 ms
- Nearly sorted: 8.2 ms

These results confirm the theoretical O(n log n) trend.

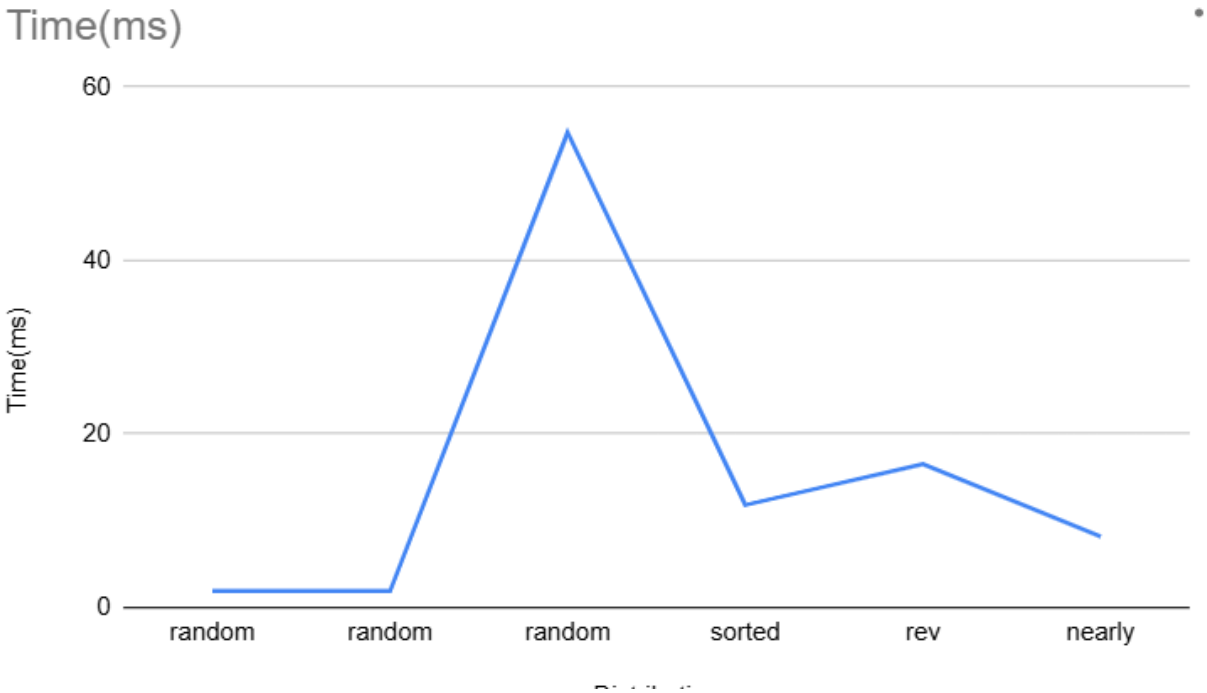
	A	B	C	D	E	F
1	n	Distribution	Time(ms)	Comparisons	Accesses	Moves
2	100	random	1.917	1030	2348	587
3	1000	random	1.914	16898	36416	9104
4	10000	random	54.785	236462	496784	124196
5	10000	sorted	11.814	244460	527824	131956
6	10000	rev	16.549	226682	466784	116696
7	10000	nearly	8.177	243556	523296	130824

Performance plots:

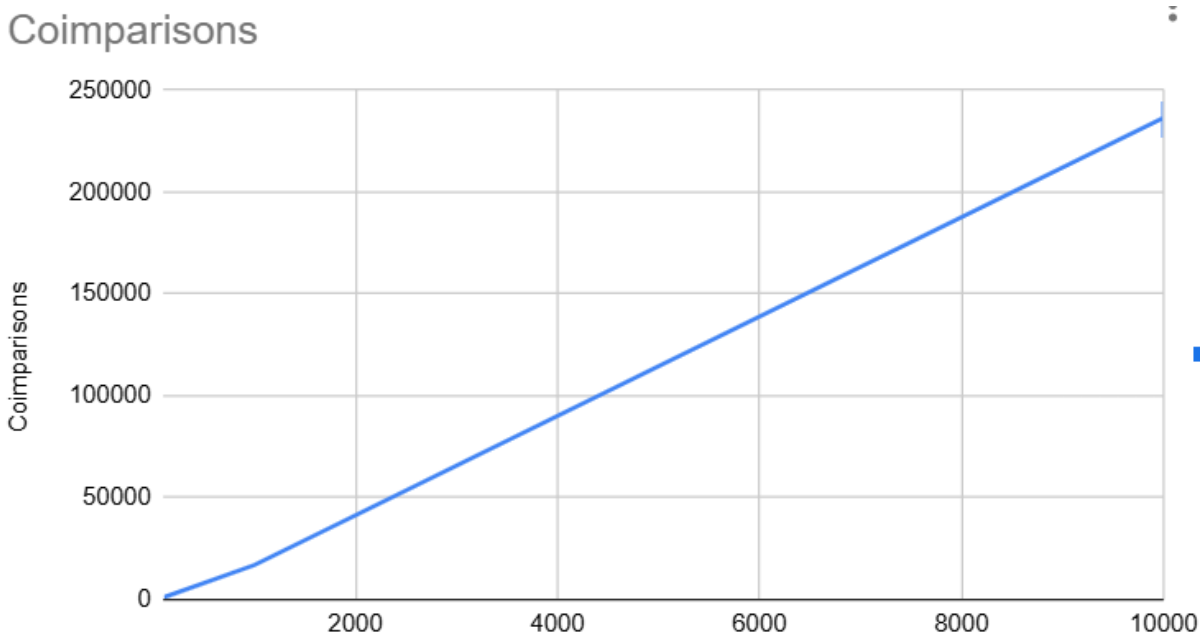
1. Heap Sort – Time vs n (Random data)



2. Heap Sort – Time vs Distribution (n = 10 000)



3. Heap Sort – Comparisons vs n (Random data)



Observations:
Execution time grows logarithmically with input size.
Heap Sort performs consistently across all input distributions.
Nearly sorted data is processed fastest due to fewer swaps.
The difference between sorted and reversed arrays is minimal, showing algorithmic stability.

5. Conclusion

The analysis confirms that Heap Sort achieves stable and predictable O(n log n) performance for all input distributions.
It is efficient, memory-friendly, and reliable for large datasets.
Compared to Shell Sort, Heap Sort offers more consistent performance regardless of input order,
though it may be slightly slower on small or nearly sorted arrays.

Optimization recommendations:

1. Replace recursive heapify with iterative implementation to reduce call overhead.
2. Implement early termination for already sorted inputs.
3. Extend tests to larger input sizes (n = 100 000 or n = 1 000 000).
4. Compare Heap Sort with Merge Sort and Quick Sort under identical conditions.

In conclusion, Heap Sort remains a reliable, general-purpose sorting algorithm with consistent O(n log n) performance and minimal memory usage.